```ocaml
(*
                              CS 51 Lab 19
                   Brian's Brain Cellular Automaton
             https://en.wikipedia.org/wiki/Brian's_Brain
 *)

module G = Graphics ;;

(* Automaton parameters *)
let cGRID_SIZE = 100 ;;   (* width and height of grid in cells *)
let cSPARSITY = 5 ;;      (* inverse of proportion of cells initially live *)

(* Rendering parameters *)
let cCOLOR_LIVE = G.rgb 200 200 200 ;;    (* color to depict live cells *)
let cCOLOR_DYING = G.rgb 130 0 0 ;;       (* color to depict dying cells *)
let cCOLOR_DEAD = G.rgb 0 0 0 ;;          (* background color *)
let cCOLOR_LEGEND = G.rgb 173 106 108 ;;  (* color for textual legend *)
let cSIDE = 8 ;;            (* width and height of cells in pixels *)
let cRENDER_FREQUENCY = 1   (* how frequently grid is rendered (in ticks) *) ;;
let cFONT = Some "-adobe-times-bold-r-normal--34-240-100-100-p-177-iso8859-9"

(*----------------------------------------------------------------------
  Brian's Brain Cellular Automaton
 *)

(*......................................................................
  States, updates, and utility functions
 *)

(* We take the opportunity to abstract the cell states as an
   enumerated type *)
type bbrain_state =
  | Live
  | Dying
  | Dead ;;

(* offset index off -- Returns the `index` offset by `off` allowing
   for wraparound. *)
let rec offset (index : int) (off : int) : int =
  if index + off < 0 then
    cGRID_SIZE - (offset ~-index ~-off)
  else
    (index + off) mod cGRID_SIZE ;;

(* bbrain_update grid i j -- Returns the updated value for cell at `i,
   j` in the grid based on rules of Brian's Brain
   <https://en.wikipedia.org/wiki/Brian%27s_Brain>. *)
let bbrain_update (grid : bbrain_state array array)
                  (i : int) (j : int)
    : bbrain_state =
  let neighbors = ref 0 in
  for i' = ~-1 to ~+1 do
```

```
    for j' = ~-1 to ~+1 do
      let i_ind = offset i i' in
      let j_ind = offset j j' in
      neighbors := !neighbors
                   + match grid.(i_ind).(j_ind) with
                     | Live -> 1
                     | Dying -> 0
                     | Dead -> 0
    done
  done;
  (* determine new state based on neighbor count *)
  let new_state =
    match grid.(i).(j) with
    | Live -> Dying
    | Dying -> Dead
    | Dead -> if !neighbors = 2 then Live else Dead in
  new_state ;;

(* bbrain_random_state () -- Returns a random cell state, either
   Live or Dead, but not Dying (as those will get generated on the
   next time step). *)
let bbrain_random_state () =
  let states = [Live; Dead] in
  List.nth states (Random.int (List.length states))

(*......................................................................
  Making Brian's Brain
 *)

(* Automaton specification *)

module BBrainSpec : (Cellular.AUT_SPEC
                     with type state = bbrain_state) =
  struct
    type state = bbrain_state
    let initial : state = Dead
    let grid_size : int = cGRID_SIZE
    let random_state = bbrain_random_state
    let update = bbrain_update
    let name : string = "Brian's Brain"
    let cell_color (cell : state) : G.color =
      match cell with
      | Live -> cCOLOR_LIVE
      | Dying -> cCOLOR_DYING
      | Dead -> cCOLOR_DEAD
    let side_size : int = cSIDE
    let legend_color : G.color = cCOLOR_LEGEND
    let font : string option = cFONT
    let render_frequency : int = cRENDER_FREQUENCY
  end ;;

module Aut = Cellular.Automaton (BBrainSpec) ;;

(*......................................................................
```

```
  Running the automaton and displaying the results
 *)

(* main seed -- Runs the automaton using the provided random `seed`
   (for replicability), displaying updates to the graphics window. *)
let main seed =

  Random.init seed;       (* initialize randomness *)
  Aut.graphics_init ();  (* ... and graphics window *)

  let initial_grid = Aut.random_grid () in
  Aut.run_grid initial_grid ;;

(* Run the automaton with user-supplied seed *)
let _ =
  if Array.length Sys.argv <= 1 then
    Printf.printf "Usage: %s <seed>\n  where <seed> is integer seed\n"
                  Sys.argv.(0)
  else
    main (int_of_string Sys.argv.(1)) ;;
```